Studio Nx

AI 구현 설명 10 — 공통 헤더·푸터·로그인 모달 통합

2026-10-12

AI 구현 설명 10 — 공통 헤더·푸터·로그인 모달 통합 (HTML 조각)

항목	내용
작성	Claude (AI)
작성일	2026-10-02
작업	여러 페이지에 반복되는 헤더, 푸터, 로그인 모달을 각각 한 파일로 통합, 이전 문서: 07_auth_pages.md §3.7
브랜치	`feature/phase1/p1-06-join-login-modal`
원칙	JS는 원래 템플릿만 만들지만, 이 불러오기 코드는 **사용자의 요청으로 AI가 구현**했습니다 (화면 조립용 기반 코드)

---

1. 무엇을 만들었나

파일	내용
`static/partials/header.html`	공통 헤더입니다 (로고, 섹션 메뉴, Sign In과 Join Us / My Page와 My Projects, 모바일 메뉴)
`static/partials/footer.html`	공통 푸터입니다
`static/partials/login-modal.html`	로그인 모달입니다 (C-2 시안, 소셜 버튼 자리)
`static/js/core/include.js`	**구현을 마쳤습니다.** `data-include` 자리를 조각 파일의 내용으로 바꿉니다
`home.html`, `join.html`	헤더, 푸터, 모달 자리를 각각 `<div data-include="...">` 한 줄로 바꿨습니다
`pages/home.js`, `pages/join.js`	맨 처음에 `await includePartials()`를 부른 뒤 다른 init을 실행합니다
`pages/join.css`	공통 헤더를 가입 페이지 모양으로 맞춥니다 (섹션 메뉴와 Join Us를 숨기고, 헤더 자리를 확보)

2. 쓰는 법

새 페이지(마이페이지 등)를 만들 때는 다음과 같이 합니다.

```
<body>
  <a class="skip-link" href="#main">Skip to content</a>
  <div data-include="static/partials/header.html"></div>
  <main id="main"> ... </main>
  <div data-include="static/partials/footer.html"></div>
  <div data-include="static/partials/login-modal.html"></div>   ← 로그인 모달이 필요할 때만
  <script type="module" src="static/js/pages/새페이지.js"></script>
</body>
```

```
// pages/새페이지.js
import { includePartials } from '../core/include.js';
await includePartials();   // 이 줄 다음부터 헤더·모달 요소를 찾을 수 있음
```

- 헤더, 푸터, 모달을 고칠 때는 **`static/partials/`의 파일 하나만** 고치면 모든 페이지에 반영됩니다.

- 페이지마다 다른 부분은 그 페이지 CSS로 숨기거나 바꿉니다 (예: `join.css`가 섹션 메뉴를 숨깁니다).

3. 왜 이렇게 했나

3.1 다른 방법과 비교 (사용자와 결정)

방법	결과
A. HTML 조각 + 작은 불러오기 JS	✅ 적용했습니다. 구조를 거의 바꾸지 않고, 마크업을 HTML 파일 그대로 관리할 수 있습니다
B. 웹 컴포넌트	마크업이 JS 문자열 안으로 들어가서 디자인을 고치기 불편합니다
C. 서버 템플릿(Jinja2)	정적 파일 제공 구조를 바꿔야 합니다
D. 빌드 스크립트	고칠 때마다 빌드해야 하고, 원본과 결과물을 이중으로 관리해야 합니다

3.2 불러오기 코드 (`include.js`)

```
[data-include] 자리를 모두 찾음
 → 동시에 fetch (Promise.all)
 → <template> 에 넣어 HTML로 해석
 → 자리(div)를 그 내용으로 교체 (div 는 남지 않음 → 이전과 같은 DOM)
```

- `<template>`을 쓰는 이유: 조각 안의 HTML을 안전하게 해석하고, 조각 안 `<script>`는 실행하지 않습니다.
- 파일을 받지 못하면 오류를 던지므로 콘솔에 바로 보입니다 (헤더가 조용히 빠지는 일을 막습니다).
- 페이지 스크립트는 `type="module"`이라 `await includePartials()`를 맨 위에서 바로 쓸 수 있습니다.

3.3 헤더 링크를 `home.html#service` 형식으로

조각은 어느 페이지에서나 쓰이므로 `#service` 대신 `home.html#service`로 바꿨습니다. `home.html` 안에서는 같은 문서라 새로고침 없이 스크롤만 되고, 다른 페이지에서는 메인 페이지의 해당 섹션으로 이동합니다.

3.4 깜빡임 대비

페이지	처리
`home.html`	헤더가 첫 화면 위에 겹쳐 있으므로(absolute) 늦게 들어와도 아래 내용이 밀리지 않습니다
`join.html`	헤더가 일반 흐름이라서, 받기 전에 자리(`min-height: var(--header-h)`)를 미리 확보합니다

측정 결과 조각 3개가 페이지 로드(DOMContentLoaded) 후 **1~7ms 안에** 끼워집니다.

3.5 알아 둘 점

- **JS가 꺼져 있으면** 헤더, 푸터, 모달이 나오지 않습니다. 로그인과 가입 자체가 JS(API 호출)로 동작하므로 이 서비스에서는 받아들일 만하다고 판단했습니다.
- `file://`로 HTML을 직접 열면 `fetch`가 막힙니다. 항상 개발 서버(`http://localhost:8000`)로 엽니다.

4. 검증 결과 (2026-10-02)

확인	결과
화면 회귀 (이전과 픽셀 비교)	`home.html` 전체 페이지(데스크톱, 모바일), 로그인 상태 헤더, 모바일 메뉴를 연 상태, 로그인 모달을 연 상태가 모두 **0픽셀 차이**입니다
가입 페이지 회귀	P1-06의 원래 캡처와 비교했고, 데스크톱과 모바일 모두 **0픽셀 차이**입니다
동작	4가지 경우(메인 헤더, 메인 모바일 메뉴, 가입 헤더, 가입 카드 아래) 모두 누르면 모달이 열리고 이메일 칸에 커서가 갑니다
남은 자리	`[data-include]`는 0개이고, 헤더와 모달은 각각 1개입니다
콘솔 오류	없습니다
디자인 자동 검사	파일, `home.html`, `join.html` 모두 0건입니다

5. 자주 받을 질문 대비

- **Q. 왜 서버 템플릿(Jinja2)을 쓰지 않았나요?** 지금은 FastAPI가 `frontend/` 폴더를 그대로 제공합니다. 템플릿으로 바꾸면 페이지마다 라우트가 생기고 구조가 커집니다. 나중에 React로 옮길 계획이 있으므로 지금 구조를 크게 바꾸지 않았습니다.
- **Q. React로 옮길 때는 어떻게 되나요?** `partials/`의 각 파일이 그대로 `<Header />`, `<Footer />`, `<LoginModal />` 컴포넌트 후보가 됩니다.
- **Q. 조각 안에 또 조각을 넣을 수 있나요?** 지금 코드는 한 단계만 처리합니다. 필요해지면 끼운 뒤 `includePartials(새 내용)`을 다시 부르면 됩니다.